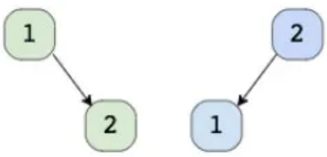
Problem Challenge 2: Structurally Unique Binary Search Trees (hard)

Problem Statement

Given a number ‘n’, write a function to return all structurally unique Binary Search Trees (BST) that can store values 1 to ‘n’?

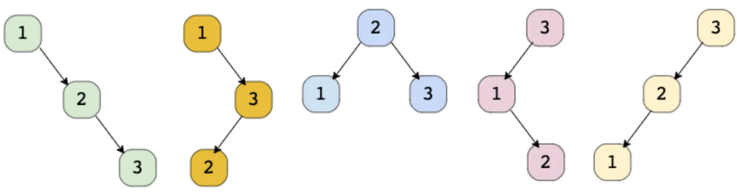
Example 1:

```
Input: 2
Output: List containing root nodes of all structurally unique BSTs.
Explanation: Here are the 2 structurally unique BSTs storing all numbers
from 1 to 2:
```



Example 2:

```
Input: 3
Output: List containing root nodes of all structurally unique BSTs.
Explanation: Here are the 5 structurally unique BSTs storing all numbers
from 1 to 3:
```



Try it yourself

Try solving this question here:

Java

```java
import java.util.*;

/*class TreeNode {
  int val;
  TreeNode left;
  TreeNode right;

  TreeNode(int x) {
    val = x;
  }
};*/

class Solution {
  public static List<TreeNode> findUniqueTrees(int n) {
    List<TreeNode> list = new ArrayList<>();
    // TODO: Write your code here
    return list;
  }
}
```

Test Case | Run Code Result | Stdout

0

Use Example Testcases | Run | Submit

← Previous
Solution: Problem Challenge 1: Evaluate Expression

Next →
Solution: Problem Challenge 2: Structurally Unique Binary Search Trees

Mark as Completed